Phelma SEOC 2A APMAN Report

Tom Hatte, Basile Hycke, Maud Zarella

September 1, 2026

Abstract

Your names must appear in alphabetical order.

You must create a Git repository on gitlab.ensimag.fr. Use any of your accounts.

The name of your repository must include your logins in alphabetical order:

`apman_login1_login2_login3`

Make sure you made your teachers members of your repository.

You may write your report in French (see line 6 in `report/preamble.tex`) or in English.

This abstract should list what you did, how, the state of your code and your broad conclusions.

Contents

1 Who did what? 5

1.1 Tom Hatte 5

1.2 Basile Hycke 5

1.3 Maud Zarella 5

List of Figures

List of Codes

Introduction

See the bibliography for a few references that may be of interest.

Chapter 1

Who did what?

1.1 Tom Hatte

1.1.1 What I did

1.1.2 What I learned

1.2 Basile Hycke

1.2.1 What I did

1.2.2 What I learned

1.3 Maud Zarella

1.3.1 What I did

1.3.2 What I learned

Bibliography

[1] David R. Hanson. *C Interfaces and Implementations: Techniques for Creating Reusable Software*. Addison-Wesley Publishing, 1996.

[2] Dennis M. Ritchie and Brian W. Kernighan. *The C Programming Language*. Bell Laboratories, 1988.

[3] Henry S. Warren. *Hacker's Delight*. Addison-Wesley Publishing, 2002.

[4] Pedro Celis, Per-Ake Larson, and Ian J. Munro. "Robin Hood Hashing". In: *Proceedings of the 26th Annual Symposium on Foundations of Computer Science*. SFCS'85. USA: IEEE Computer Society, 1985, pp. 281–288. ISBN: 0818608444. DOI: 10.1109/SFCS.1985.48.

[5] Ulrich Drepper. *What Every Programmer Should Know About Memory*. Tech. rep. RedHat, 2007.

[6] Sean E. Anderson. *Bit-Twiddling Hacks*. Ed. by Stanford University. 1997. URL: https://graphics.stanford.edu/~seander/bithacks.html (visited on 08/10/2025).

[7] Wikibooks Contributors. *C Programming*. Ed. by Wikibooks. URL: https://en.wikibooks.org/wiki/C_Programming (visited on 08/10/2025).

[8] Programming Guide Contributors. *Robin Hood Hashing*. Ed. by Programming Guide. URL: https://programming.guide/robin-hood-hashing.html (visited on 08/10/2025).

[9] Wikipedia, ed. *Double-Ended Queue*. URL: https://en.wikipedia.org/wiki/Double-ended_queue (visited on 08/10/2025).

[10] Pat Morin. *Open Data Structures In Pseudocode*. URL: https://opendatastructures.org/ods-python/ods-python-html.html (visited on 08/10/2025).

[11] Simon Tatham. *Coroutines in C*. 2000. URL: http://www.chiark.greenend.org.uk/~sgtatham/coroutines.html (visited on 08/10/2025).

[12] Ryan Fleury. *Untangling Lifetimes: The Arena Allocator*. 2022. URL: https://www.rfleury.com/p/untangling-lifetimes-the-arena-allocator (visited on 08/10/2025).

[13] Chris Wellons. *Arena Allocator Tips and Tricks*. 2023. URL: https://nullprogram.com/blog/2023/09/27/ (visited on 08/10/2025).

[14] Chris Wellons. *An Easy-to-Implement, Arena-friendly Hash Map*. 2023. URL: https://nullprogram.com/blog/2023/09/30/ (visited on 08/10/2025).

[15] Allan Jackson. *Convenient Containers*. Ed. by GitHub. 2023. URL: https://github.com/JacksonAllan/CC (visited on 08/10/2025).

[16] Allan Jackson. *Verstable*. Ed. by GitHub. 2023. URL: https://github.com/JacksonAllan/Verstable (visited on 08/10/2025).

[17] Nicolas Frattaroli. *Linked Lists in Linux*. URL: https://docs.kernel.org/next/core-api/list.html (visited on 08/10/2025).

[18] GNU GlibC Contributors. *Malloc Internals*. 2022. URL: https://sourceware.org/glibc/wiki/MallocInternals (visited on 08/10/2025).

[19] David A. Wheeler. *Program Library HOWTO*. Ed. by The Linux Documentation Project. 2003. URL: https://tldp.org/HOWTO/Program-Library-HOWTO/index.html (visited on 08/10/2025).

[20] Gabriel G. Cunha. *Exploring How Cache Memory Really Works*. 2024. URL: https://pikuma.com/blog/understanding-computer-cache (visited on 08/10/2025).

[21] Michael E. Lee. *Optimization of Computer Programs in C*. 1997. URL: https://icps.u-strasbg.fr/~bastoul/local_copies/lee.html (visited on 08/10/2025).

[22] David Drysdale. *How Programs Get Run: ELF Binaries*. Ed. by LWN. 2015. URL: https://lwn.net/Articles/631631/ (visited on 08/10/2025).

[23] Erik Corry. *Hash Maps That Don't Hate You*. 2021. URL: https://medium.com/the-toit-take/hash-maps-that-dont-hate-you-1a96150b492a (visited on 08/10/2025).

[24] Malte Skarupke. *Fibonacci Hashing: The Optimization that the World Forgot*. 2018. URL: https://probablydance.com/2018/06/16/fibonacci-hashing-the-optimization-that-the-world-forgot-or-a-better-alternative-to-integer-modulo/ (visited on 08/10/2025).

[25] Daniel Lemire. *A Fast Alternative to the Modulo Reduction*. 2016. URL: https://lemire.me/blog/2016/06/27/a-fast-alternative-to-the-modulo-reduction/ (visited on 08/10/2025).

[26] Thomas Neumann. *All Hash Table Sizes You Will Ever Need*. 2020. URL: https://databasearchitects.blogspot.com/2020/01/all-hash-table-sizes-you-will-ever-need.html (visited on 08/10/2025).

[27] Piotr Duperas. *NaN Boxing or How to Make the World Dynamic*. 2020. URL: https://piotrduperas.com/posts/nan-boxing (visited on 08/10/2025).

[28] Robert Nystrom. *Crafting Iinterpreters*. 2015. URL: https://craftinginterpreters.com/ (visited on 08/10/2025).

[29] Basile Starynkevitch. *Answer to Question 429188*. Ed. by Stack Exchange. 2018. URL: https://unix.stackexchange.com/a/429209 (visited on 08/10/2025).

[30] Contributors. *Treaps (Cartesian Trees)*. Ed. by Algorithms for Competitve Programming. 2022. URL: https://cp-algorithms.com/data_structures/treap.html (visited on 08/10/2025).

[31] Scot Drysdale. *CS10: Circular, Doubly Linked Lists with Sentinel*. Ed. by Dartmouth University. 2016. URL: https://www.cs.dartmouth.edu/~scot/cs10/lectures/6/6.html#DLL (visited on 08/10/2025).